

Bot-DCA

Permissionless Dollar-Cost Averaging on BOT Chain

A non-custodial savings automation protocol: lock a budget once, and an open keeper network executes your recurring BOT⇌USDT swaps — every fill provable on-chain, no operator ever touching user funds.

LIVE ON BOT CHAIN TESTNET

Contract (testnet)	0x6eB819d09EFAF3Eb3ce52C4D6db3da6B2Fa37895 (verified on BOTScan)
Chain	BOT Chain Testnet · chain ID 968 · 0.75s blocks · ~\$0.06 gas
Stack	Solidity ≥0.8.20 (Hardhat) · Node/viem keeper · React + wagmi dApp
Repositories	github.com/FrancisSix/Bot-DCA
Status	Deployed, verified, end-to-end tested with an autonomous keeper

1 · Abstract

Bot-DCA turns dollar-cost averaging — buying a fixed amount of an asset on a fixed schedule — into a smart-contract primitive on BOT Chain. A user deposits a budget once into the **BotDCA** contract, specifying an amount per interval, an interval length, a number of intervals, and a slippage tolerance. An open, permissionless keeper network monitors due positions and executes the swaps through the chain's BDEX Uniswap V2/V3 venues, earning a small fee (0.1%) per fill. The contract holds the funds, enforces every constraint, and credits output tokens to the position. Nobody — not the team, not a keeper — can misappropriate deposits, censor execution, or alter terms after creation.

2 · Problem

Recurring-buy automation exists on centralized exchanges, but on-chain DCA faces three structural problems:

1. Custody risk

Centralized auto-invest requires depositing funds with an operator who controls timing and custody.

2. Automation is privileged

Typical on-chain automation relies on a single operator's server; if it stops, your strategy stops.

3. Thin-liquidity pricing

On a young chain, AMM pools drift; naive DCA either overpays or reverts without on-chain price guards.

4. Opaque execution

Users can't verify that scheduled buys executed at fair prices without reading raw events.

Bot-DCA addresses all four: funds are held by the contract itself; *anyone* can execute a due position (so automation has no single point of failure); minimum-output amounts are computed on-chain from live pool state; and every fill is recorded as an on-chain execution record with its exact input/output amounts.

3 · Protocol Design

3.1 Positions

A position is created with `createPosition(tokenIn, tokenOut, amountPerInterval, intervalSeconds, numIntervals, slippageBps)`, paying the full budget up front (native BOT via `msg.value`, or USDT via `transferFrom`). The contract validates terms against on-chain bounds (min interval, max intervals, max slippage) and stores the position; creation is atomic and non-custodial by construction.

3.2 Permissionless execution

Any address may call `execute(id)` (or `executeWithPath(id, path)` for multi-hop routes) once a position is due. The contract itself:

- verifies the position is active, incomplete, and due;
- computes a **minimum output** — from the V3 pool's live `slot0` price when using V3, or the V2 router's quoted path output — scaled by the user's slippage tolerance;
- swaps exactly `amountPerInterval` through the configured venue;
- deducts the keeper fee (0.1% of output) and credits the remainder to the position's accumulator;
- writes an immutable execution record (`timestamp, amountIn, amountOut`) and marks completion after the final interval.

Because execution is permissionless, keepers compete redundantly: multiple independent keepers can run simultaneously without coordination, and a stalled keeper cannot strand a position.

3.3 Execution venue

The default venue is BDEX V3 (0.30% WBOT/USDT pool), selected after observing material staleness in the V2 pool on testnet. The venue and fee tier are owner-tunable parameters, and V3 output in native BOT is unwrapped from WBOT inside the same transaction. Native input is wrapped via the router; the contract never holds a stale approval beyond what each swap needs.

3.4 User exits

- `withdraw(id)` — claim accumulated output tokens at any time;
- `cancel(id)` — refund the unspent budget pro-rata (plus any accrued output) and close the position.

4 · System Architecture

User (React + wagmi dApp)

→ **create / approve / withdraw / cancel**

BotDCA contract

Keeper(s) · Node + viem

→ **poll isDue** → **simulate** → **execute** → **fee**

BDEX V2 / V3 routers

Windows scheduled-task supervisor ships in-repo: auto-start at logon, 10-min cadence check, mutex-guarded so only one supervisor instance runs.

Contracts (Solidity / Hardhat)

BotDCA core with OpenZeppelin
ReentrancyGuard + Ownable; interfaces for
Uniswap V2 router and V3
SwapRouter/factory/pool. 10/10 tests passing
against mocked venues.

Keeper (Node + viem)

Polls `nextId / isDue`, simulates each execution
before sending, retries with backoff, logs a
heartbeat. Fully stateless — safe to run many
in parallel.

dApp (React + wagmi + Vite)

Create/manage positions, live V3-derived
pricing, per-position execution history. Pre-
flight checks: network guard, contract-
presence check, decimal-correct amounts.

Operations

One-command deploy + verification scripts;
explorer-verified sources; read-only inspection
tooling; E2E script that drives a real position to
completion.

4.1 Security model

- **No custody pathway:** the contract has no admin transfer of user funds; owner functions tune parameters only (bounds, fee $\leq 1\%$, venue, pause).
- **Reentrancy-guarded** state changes; swaps and accounting are atomic per interval.
- **Slippage enforcement on-chain:** a fill that cannot meet the user's minimum output reverts — the user's budget is never spent at an arbitrarily bad price.
- **Redundant keepers:** execution rights are open; the fee (0.1%) is the only incentive needed to sustain the network.
- **Pause switch:** a global pause exists for incident response; it cannot move funds, only halt new activity.

5 · Live Testnet Results

The full loop was exercised on BOT Chain testnet with an autonomous keeper and no manual intervention between creation and completion:

POSITION	STRATEGY	RESULT
#3	Buy 0.5 BOT × 3 @ 10s intervals (BOT→USDT)	3/3 executed at t+17s/34s/46s; accrued 15.4625 USDT; avg \$10.3083/BOT vs market \$10.3186 (−0.10%)
#5	Buy 200 USDT × 3 @ 10s (USDT→BOT)	3/3 executed; received ≈19.2470 BOT per interval (≈ \$10.39/BOT effective)
#1/#4/#6	1–5 BOT × 3 @ 10s (BOT→USDT)	All completed; per-interval outputs within tight variance of spot

Interpretation: a fresh wallet can deposit once and watch a complete multi-interval strategy execute itself in under a minute, with realized prices within 0.1% of the V3 market price and full on-chain auditability of every fill.

5.1 Contract parameters (live, testnet)

minInterval	10 seconds (testnet setting for fast iteration)
maxIntervals	1,000
maxSlippageBps	1,000 (10%) ceiling; user chooses per position
keeperFeeBps	10 (0.1% of output per fill)
paused	false
Default venue	V3, fee tier 0.30%

5.2 Roadmap

- **Now:** testnet-hardened contracts, keeper supervision, dApp with live pricing and execution history.
- **Next:** mainnet (chain 677) deployment; keeper fee auctions; multi-hop V3 routes; top-up without recreating.
- **Later:** additional output assets beyond USDT; strategy templates (buy-the-dip modifiers); indexer-backed analytics UI.

6 · Getting Started

- dApp: any Vercel deployment of `frontend/` (auto-deploys from `main`)
- Fund a wallet with testnet BOT (faucet: `faucet.botchain.ai`); USDT optional, per direction
- Run your own keeper: `keeper/` — `npm i && npm run start`, or install the provided Windows scheduled task
- Contracts: `contracts/` — Hardhat tests, deploy, verify, E2E scripts included

7 · Conclusion

DCA is the most common investing habit in the world, and it deserves an on-chain form that is trust-minimized end to end: budget in the contract, terms immutable at creation, execution open to anyone, prices guarded on-chain, and every fill auditable forever. Bot-DCA demonstrates that this is practical today on BOT Chain — a verified contract, an open keeper that already runs unattended, and a dApp that makes the whole lifecycle visible. The protocol is live on testnet and ready for mainnet.

Repo	github.com/FrancisSix/Bot-DCA
Explorer	scan.bohr.life/address/0x6eB819d09EfaF3Eb3ce52C4D6db3da6B2Fa37895
Contact	GitHub issues · FrancisSix

This document describes software deployed to a public testnet for evaluation. Nothing here is financial advice; DCA strategies can lose value. Parameters quoted are live testnet values as of September 30, 2026.